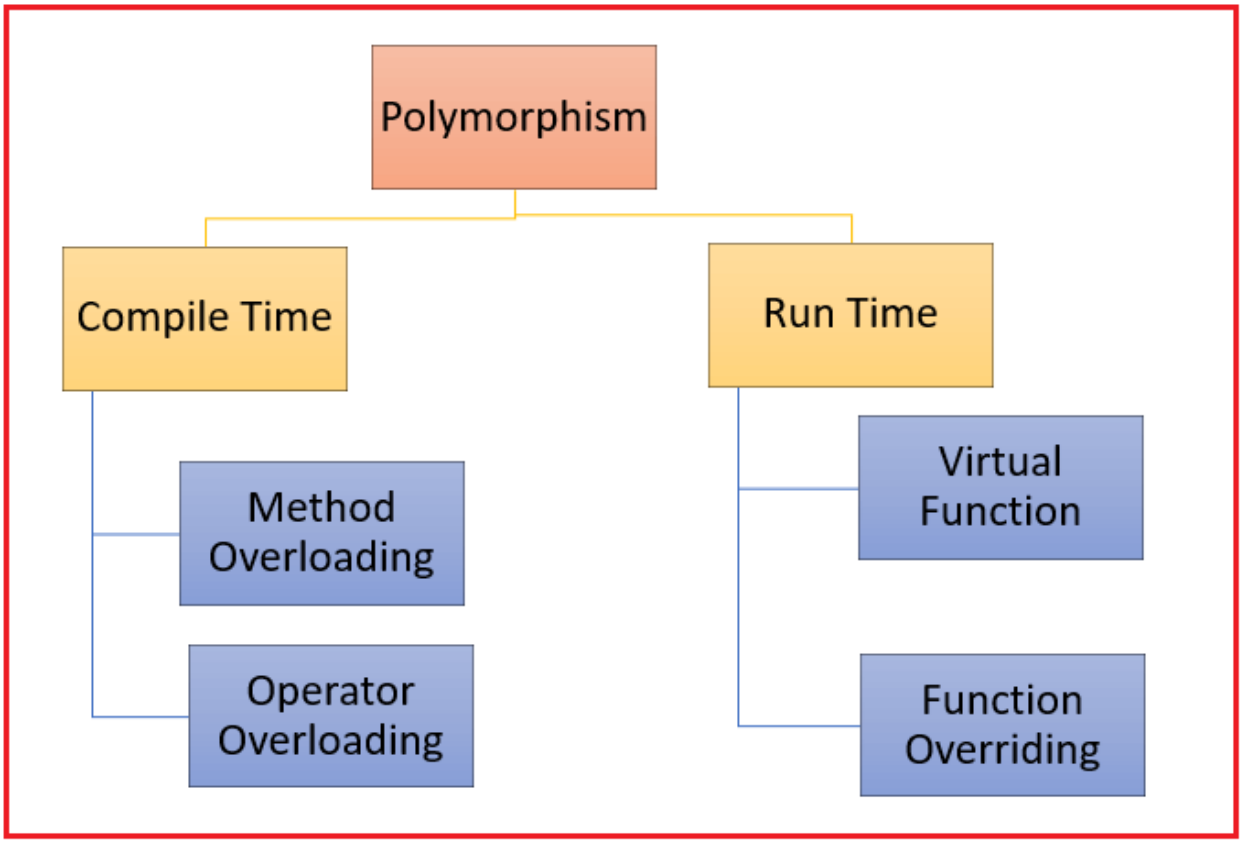
Polymorphism

Polymorphism is considered one of the important features of Object-Oriented Programming. The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.

One real-life example of polymorphism is, that a person at the same time can have different characteristics. A man at the same time is a father, a husband, and an employee. So, the same person possesses different behavior in different situations. This is called polymorphism.

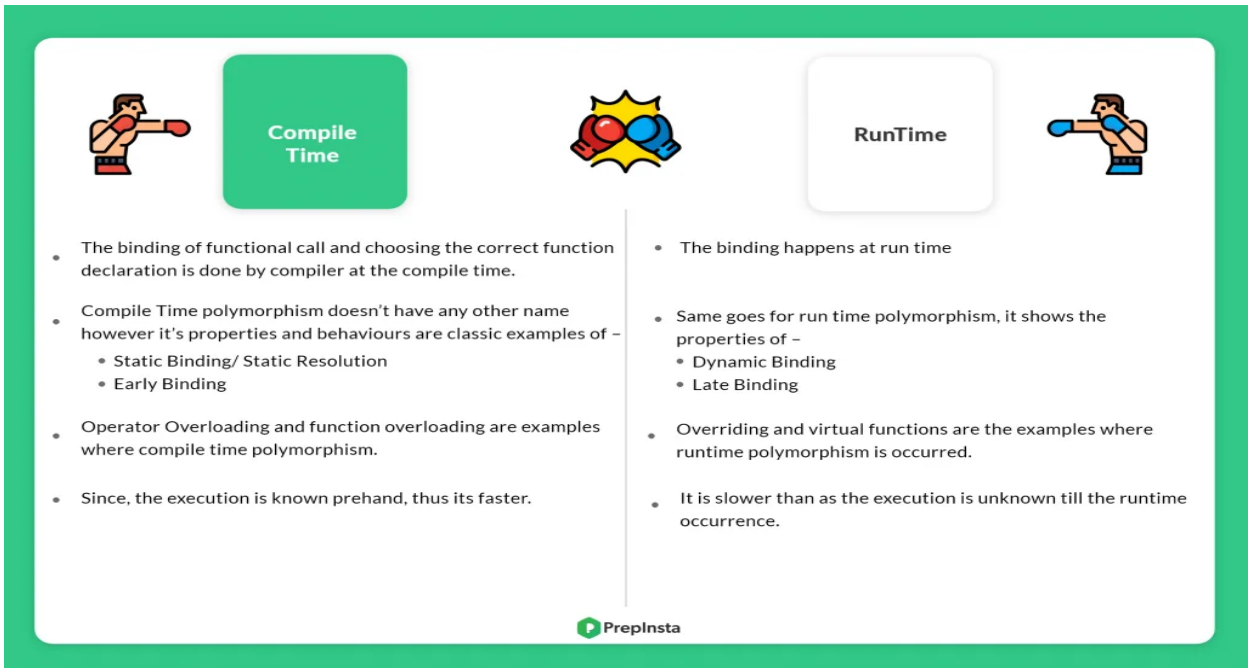


Compile Time Polymorphism in C++ :- This type of polymorphism is achieved by **function overloading** or **operator overloading**. The overloaded functions are invoked by matching the type and number of arguments.

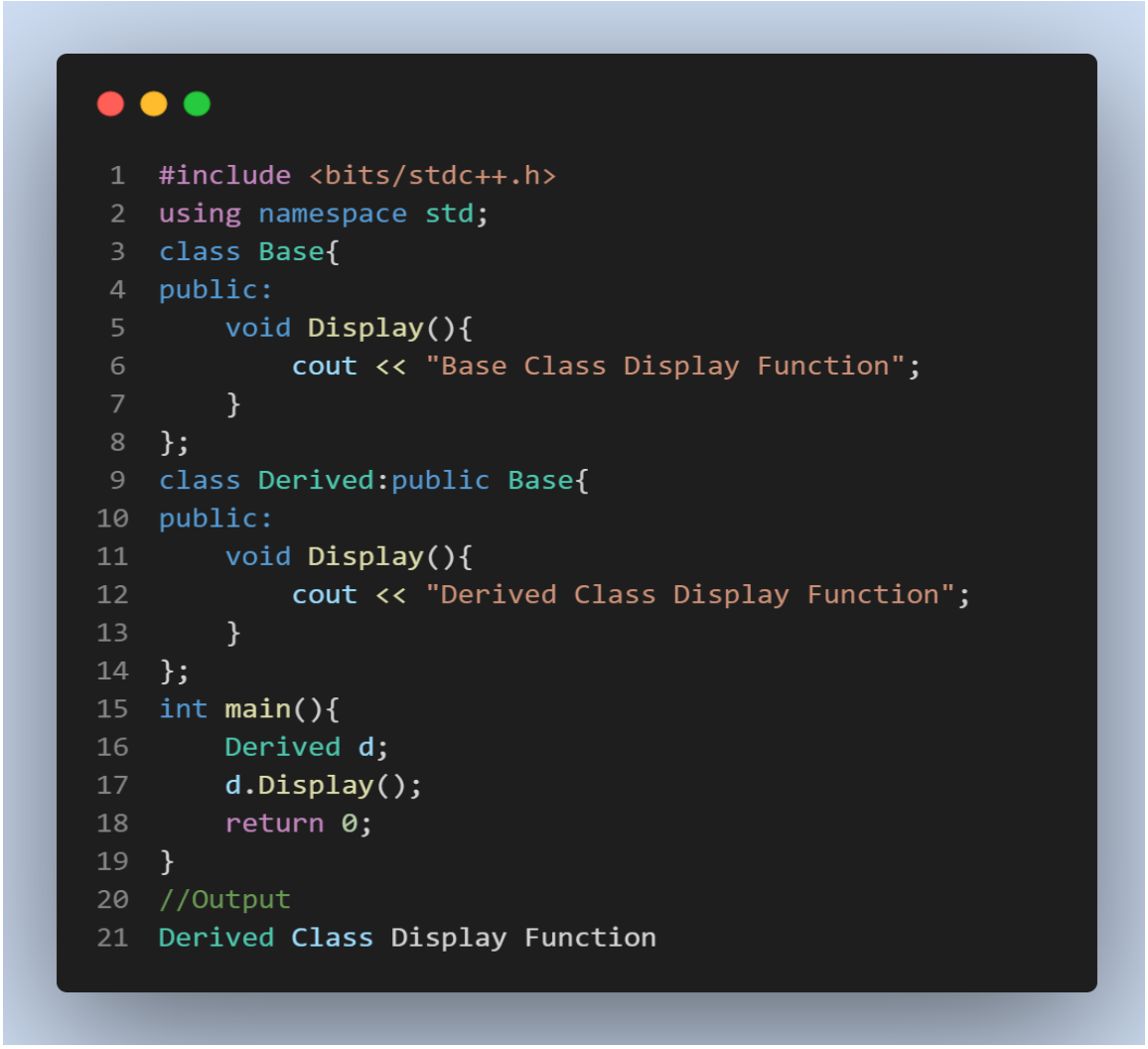
The information is present during compile-time. This means the C++ compiler will select the right function at compile time. Which is also known as static binding or early binding.

Runtime Polymorphism in C++ :- Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by method overriding which is also known as dynamic binding or late binding.

This type of polymorphism is achieved by Function Overriding. This happens when an object's method is invoked/called during runtime rather than during compile time.



Function overriding -



```cpp
#include <bits/stdc++.h>
using namespace std;
class Base{
public:
    void Display(){
        cout << "Base Class Display Function";
    }
};
class Derived:public Base{
public:
    void Display(){
        cout << "Derived Class Display Function";
    }
};
int main(){
    Derived d;
    d.Display();
    return 0;
}
//Output
Derived Class Display Function
```

Key Points about Function overriding :-

- Redefining a function of a base class in the derived class called function overriding in c++.
- Function overriding is used for achieving runtime polymorphism.
- The Signature and Prototype of an overrides function must be exactly the same as the base function.

When do we need to override a function in C++?

If the superclass function logic is not fulfilling the sub-class business requirements, then the subclass needs to override that function with the required business logic. Usually, in most real-time applications, the superclass functions are implemented with generic logic which is common for all the next-level sub-classes.

Note :- If a function in the sub-class contains the same signature as the superclass non-private function, then the subclass function is treated as the **overriding function** and the superclass function is treated as the **overridden function.**

Virtual Function in C++ :- A virtual function in C++ is a member function that is declared within a base class using the virtual keyword and is re-defined by a derived class. When we refer to a **derived class object using the base class pointer** or base class reference variable, and when we can call a virtual function, then it will execute the function from the derived class.

So, Virtual Functions in C++ ensure that the correct function is called for an object, regardless of the expression used to make the function call.

```cpp
#include<bits/stdc++.h>
using namespace std;
class Base{
public:
    void Display(){
        cout << "Display of Base" << endl;
    }
};
class Derived:public Base{
    public:void Display(){
        cout << "Display of Derived " << endl;
    }
};
int main(){
    Base *p = new Derived ();
    p->Display ();
}
//Output
Display of Base
```

```cpp
#include<bits/stdc++.h>
using namespace std;
class Base{
public:
    virtual void Display(){
        cout << "Display of Base" << endl;
    }
};
class Derived:public Base{
    public:void Display(){
        cout << "Display of Derived " << endl;
    }
};
int main(){
    Base *p = new Derived ();
    p->Display ();
}
//Output
Display of Derived
```